

Q-ADAPTIVE AI Guardian

Yapay Zekâ Güdümlü, Kuantum Dirençli
Uyarlanabilir Akıllı Cüzdan

SIFIRDAN ÖĞRETİCİ BELGE

Hiç bilmeyen bir okuyucu için, kavram kavram, adım adım

Takım: CryptoTEK · TAKIM ID 909630
Başvuru ID 2603893

TEKNOFEST 2026 — Blokzincir Yarışması
Final: 30 Eylül – 4 Ekim 2026, Şanlıurfa

İçindekiler

Belge, hiçbir ön bilgi varsaymayan bir sırayla ilerler. Her kavram, kullanılmadan önce tanımlanır. Bir bölümü atlarsanız sonraki bölüm eksik kalabilir.

#	Bölüm	Ne öğreneceksiniz
0	Bu belge nedir	Belgenin amacı, kimin için yazıldığı ve sayıların nasıl doğrulandığı
1	Proje nedir, neden var	Hangi yarışma, hangi problem, tek cümlelik değer önermesi
2	Çözülen problem	Kuantum tehdidi ve cüzdan güvenliğinin bugünkü açıkları
3	Temel kavramlar	Blozkincirden ZK-STARK'a kadar 11 kavram, sıfırdan
4	Mimari: dört katman	Parçaların birbirine nasıl bağlandığı
5	Katmanlar ayrıntılı	AI, ZK, sözleşme ve arayüz katmanlarının iç işleyişi
6	Uçtan uca bir koşu	Bir işlemin 11 aşamadan nasıl geçtiği, ölçülen sürelerle
7	Arayüz	Ekran görüntüleriyle, her bölümün ne gösterdiği
8	Neler yapabiliyor	Kanıtlanmış yetenekler ve bunları koruyan testler
9	Neler yapamıyor	Eksikler, mimari sınırlar ve her birinin sebebi
10	Test ve doğrulama	244 otomatik test, kapsam ölçümleri, CI hattı
11	Denetim hikâyesi	Ne bulundu, nasıl düzeltildi, hangi ders çıktı
12	Çalıştırma kılavuzu	Sıfırdan kurulum ve doğrulama komutları
13	Sözlük	Belgede geçen tüm terimlerin kısa tanımları

0. Bu belge nedir

Bu belge, Q-ADAPTIVE adlı projeyi hiç bilmeyen birine baştan sona anlatmak için yazıldı. Blokzincirin ne olduğunu bilmiyorsanız bile takip edebilirsiniz: her kavram, ilk kullanıldığı yerde tanımlanır ve neden gerektiği açıklanır.

Kimin için

- Jüri üyeleri — sistemin ne yaptığını ve neyi yapmadığını tek kaynaktan görmek isteyenler.
- Ekip üyeleri — kendi katmanı dışındaki parçaları anlamak isteyenler.
- Projeyi devralacak kişiler — kodun neden böyle yazıldığını merak edenler.
- Meraklı okuyucular — post-kuantum kriptografi ve sıfır bilgi kanıtlarının pratikte nasıl birleştiğini görmek isteyenler.

Belgedeki sayılar nasıl doğrulandı

Bu projenin denetiminde çıkan en büyük sorun şuydu: belgeler "ML-DSA kullanıyoruz" diyordu ama kodda tek satır ML-DSA yoktu. Aynı hataya düşmemek için bu belgede üç kural uygulanır:

- Her sayı koddan okunmuştur ya da bir komut çalıştırılarak ölçülmüştür. Tahmin yok.
- Ölçülmemiş bir şey "ölçülmedi" diye yazılır. Makul bir tahmin yazmak, ölçmüş gibi görünmek demektir.
- Yapılmamış bir şey "yapılmadı" diye yazılır. "İleride yapılacak" ifadesi, mevcut durumun yerine değil, ardına konur.

Donanıma bağlı değerler. Süreler (çıkartım, kanıt üretimi) ve kuyruk kapasitesi gibi değerler çalıştığınız makineye göre değişir. Bu belgede aralık verilir ve hangi makinede ölçüldüğü söylenir. Sabit bir sayı görürseniz, o sayı gerçekten sabittir (örneğin bir imzanın bayt uzunluğu).

1. Proje nedir, neden var

Tek cümlelik özet

Q-ADAPTIVE, bir kripto cüzdanının güvenlik seviyesini tehdide göre canlı olarak değiştiren bir sistemdir: yapay zekâ işlem davranışında anomali görürse, cüzdan kendini otomatik olarak daha güçlü, kuantum bilgisayarlara dirençli bir imza şemasına yükseltir ve bu geçişin gerçekten olduğunu matematiksel bir kanıtla belgeler.

Hangi yarışma

Alan	Değer
Yarışma	TEKNOFEST 2026 — Blokzincir Yarışması
Takım	CryptoTEK
Takım ID	909630
Başvuru ID	2603893
Final	30 Eylül - 4 Ekim 2026, Şanlıurfa
Proje adı	Q-ADAPTIVE (AI Guardian)
Lisans	Apache 2.0

Neden "uyarlanabilir"

Bugünkü cüzdanlar statiktir. Bir cüzdan hangi imza şemasını kullanıyorsa, ömrü boyunca onu kullanır. Tehdit ortamı değişse de cüzdan değişmez. Bu, güvenliği tek bir noktada dondurur.

Q-ADAPTIVE bunun yerine üç kademeli bir zırh kullanır ve kademeyi çalışma anında seçer. Tehdit yoksa en hafif kademe, tehdit varsa en ağır kademe. Bu yaklaşımın literatürdeki adı Hareketli Hedef Savunmasıdır (Moving Target Defense) ve 3. bölümde ayrıntılı anlatılacaktır.

Projenin üç iddiası

İddia	Nasıl karşılanıyor
Tehdidi görebilir	Zincir dışı bir yapay zekâ modeli işlem davranışını puanlar ve dinamik bir eşikle karşılaştırır.
Kendini güçlendirebilir	Puan eşiği aşarsa imza şeması NIST FIPS 204 ML-DSA'nın daha yüksek kademesine çıkar; imza 2.420 → 3.309 → 4.627 bayta büyür.
Bunu kanıtlayabilir	Geçişe eşlik eden kafes ilişkisi için bir ZK-STARK kanıtı üretilir ve zincire bir guardian imzasıyla çapalanır.

Üçüncü iddianın sınırı şimdiden söylensin: üretilen kanıt, ML-DSA imzasının geçerliliğini kanıtlamaz. Neyi kanıtladığı 3.8 ve 9. bölümlerde tam olarak açıklanır. Bu ayrım projenin en çok yanlış anlaşılan noktasıdır ve bilerek en başta belirtilmektedir.

2. Çözölmeye çalışılan problem

2.1 Bugünkü cüzdanların iki açığı

Bir kripto cüzdanı, özünde bir anahtar çiftidir. Özel anahtar sizde durur ve işlemleri imzalar; açık anahtardan türetilen adres herkese açıktır. Bu yapının iki temel zayıflığı var:

Açık 1 — Tek noktadan kırılma

Özel anahtar çalınırsa her şey biter. Saldırgan cüzdandaki tüm varlıkları tek bir işlemle boşaltabilir. Buna sektörde "drainer" saldırısı denir. Klasik cüzdanda bunu durduracak bir ara katman yoktur: imza geçerliyse işlem geçerlidir.

Açık 2 — Kuantum tehdidi

Bugün neredeyse tüm blokzincirler ECDSA adlı imza şemasını kullanır. ECDSA'nın güvenliği, eliptik eğri üzerinde ayrık logaritma probleminin zorluğuna dayanır. Yeterince büyük bir kuantum bilgisayar, Shor algoritması ile bu problemi verimli biçimde çözer — yani açık anahtardan özel anahtarı hesaplayabilir.

"Şimdi topla, sonra çöz" saldırısı. Tehdit gelecekte değil, bugün başlıyor. Bir saldırgan bugün zincirdeki açık anahtarları kaydedebilir ve kuantum bilgisayar geldiğinde geçmişe dönük olarak özel anahtarları çözebilir. Blokzincir verisi kalıcı ve herkese açık olduğu için, bugün yapılan bir işlem gelecekte kırılabilir. Bu yüzden geçişin kuantum bilgisayar gelmeden önce yapılması gerekir.

2.2 Neden basitçe "hep en güçlü imzayı kullan" demiyoruz

Akla gelen ilk çözüm şudur: kuantum dirençli imzayı her zaman kullan. Sorun maliyettir.

İmza şeması	İmza boyutu	Anlamı
ECDSA (secp256k1)	65 bayt	Bugünkü standart, kuantuma karşı kırık
ML-DSA-44	2.420 bayt	ECDSA'nın ~37 katı
ML-DSA-65	3.309 bayt	ECDSA'nın ~51 katı
ML-DSA-87	4.627 bayt	ECDSA'nın ~71 katı

Blokzincirde her bayt para demektir (bu maliyete "gaz" denir; 3.1'de anlatılacak). İşlem başına 4.627 bayt taşımak, cüzdanı pratikte kullanılamaz hâle getirir. Üstelik kullanıcıların büyük çoğunluğu, çoğu zaman saldırı altında değildir.

2.3 Q-ADAPTIVE'in cevabı

Fikir şu: her zaman değil, gerektiğinde. Sistem normal koşullarda hafif kademede çalışır; davranışsal bir anomali görüldüğünde ağır kademeye yükselir. Ve bu yükselmenin gerçekten olduğunu bağımsız olarak doğrulanabilir bir kanıtla belgeler.

Bunu yapabilmek için dört şey gerekir ve projenin dört katmanı tam olarak bunlara karşılık gelir:

- Tehdidi görececek bir şey → yapay zekâ katmanı
- Zırhı değiştirecek bir şey → post-kuantum kripto katmanı

- Değişimi kanıtlayacak bir şey → ZK-STARK katmanı
- Kararı uygulayacak bir şey → akıllı sözleşme katmanı

3. Temel kavramlar — sıfırdan

Bu bölüm, sistemin anlaşılması için gereken her kavramı tanımlar. Bir kavramı biliyorsanız o başlığı atlayabilirsiniz; bilmiyorsanız sonraki bölümler anlamsız gelecektir.

3.1 Blokzincir, işlem ve gaz

Blokzincir, binlerce bilgisayarın aynı kaydın kopyasını tuttuğu, merkezî bir otoritesi olmayan bir veritabanıdır. Kayıt yalnızca eklenebilir; geçmiş değiştirilemez.

İşlem (transaction), bu kayda yazılan bir komuttur: "şu adresten bu adrese şu kadar gönder" gibi. Her işlem, gönderenin özel anahtarıyla imzalanır; ağdaki herkes imzayı açık anahtarla doğrulayabilir.

EVM (Ethereum Sanal Makinesi), zincir üstünde program çalıştıran sanal bilgisayardır. Bu programlara akıllı sözleşme denir ve genelde Solidity dilinde yazılır.

Gaz (gas), EVM'de yapılan her işin ücretidir. Toplama gibi basit bir işlem ucuz, veri depolamak pahalıdır. Kullanıcı bu ücreti ağın yerel parasıyla (Ethereum'da ETH) öder. Zincire yazılan veriye calldata denir ve bayt başına ücretlendirilir — bu yüzden imza boyutu doğrudan maliyettir.

3.2 Dijital imza ve ECDSA

Dijital imza üç işlemden oluşur:

- Anahtar üretimi: bir özel anahtar ve ona karşılık gelen bir açık anahtar üretilir.
- İmzalama: özel anahtar + mesaj → imza.
- Doğrulama: açık anahtar + mesaj + imza → doğru/yanlış.

ECDSA, eliptik eğri matematiğine dayanan yaygın bir imza şemasıdır. Ethereum'da ecrecover adlı yerleşik bir fonksiyon, imzadan imzalayanın adresini geri çıkarır ve bu çok ucuzdur (~3.000 gaz). Projede guardian imzaları hâlâ ECDSA ile doğrulanır — nedeni 5.3'te açıklanacaktır.

3.3 Kuantum tehdidi ve Shor algoritması

Klasik bir bilgisayar, eliptik eğri ayrık logaritma problemini çözmek için astronomik sürelerle ihtiyaç duyar. Shor algoritması ise yeterince büyük bir kuantum bilgisayarda bu problemi polinom zamanda çözer. Pratik sonucu: ECDSA'nın açık anahtarından özel anahtar hesaplanabilir hâle gelir.

Yaygın bir karışıklık. Kuantum bilgisayarlar her şeyi kırmaz. Hash fonksiyonlarına (SHA-256, BLAKE3) karşı yalnızca Grover algoritması vardır ve o da güvenliği yarıya indirir — 256 bit hash, kuantum karşısında 128 bit gibi davranır ki bu hâlâ güvenlidir. Asıl kırılan, ECDSA ve RSA gibi açık anahtarlı şemalardır.

3.4 Post-kuantum kriptografi ve kafesler

Post-kuantum kriptografi, kuantum bilgisayarların da çözemediği matematiksel problemlere dayanan şemaların genel adıdır. En olgun aile kafes tabanlı (lattice-based) olanlardır.

Kafes nedir: Bir kafes, uzayda düzenli aralıklarla dizilmiş noktalar kümesidir — sonsuza uzanan üç boyutlu bir ızgara gibi düşünün, ama boyut sayısı yüzlerce. Bu yapıda "verilen bir noktaya en yakın kafes noktasını bul" gibi problemler, boyut arttıkça hem klasik hem kuantum bilgisayarlar için çok zorlaşır.

MLWE (Module Learning With Errors): Projenin dayandığı problem budur. Sezgisi şöyledir: bir A matrisi ve küçük katsayılı gizli bir s_1 vektörü alın, çarpın, üstüne küçük bir hata vektörü s_2 ekleyin:

$$t = A \cdot s_1 + s_2 \pmod{q}$$

A ve t herkese açıktır; s_1 ve s_2 gizlidir. Hata terimi olmasaydı s_1 'i lineer cebirle bulmak kolay olurdu. Hata, problemi kafeste "en yakın noktayı bulma" problemine çevirir ve bilinen hiçbir kuantum algoritması bunu verimli çözmez.

Projede $q = 8.380.417$ kullanılır; bu, ML-DSA standardının kendi asal modülüdür.

3.5 ML-DSA (FIPS 204) — kullanılan post-kuantum imza

NIST (ABD Ulusal Standartlar ve Teknoloji Enstitüsü) yıllar süren bir yarışma sonunda post-kuantum standartlarını seçti. İmza standardı FIPS 204 olarak yayımlandı ve şemanın adı ML-DSA'dır (Module-Lattice-Based Digital Signature Algorithm). Yarışmadaki adı CRYSTALS-Dilithium olduğu için iki isim birlikte kullanılır.

ML-DSA'nın üç güvenlik kademesi vardır. Projede üçü de kullanılır ve kademe çalışma anında seçilir:

Kademe	NIST kategorisi	Kafes ($k \times l$)	İmza	Açık anahtar	Gizli anahtar
ML-DSA-44	2	$4 \times 4 = 16$ hücre	2.420 B	1.312 B	2.560 B
ML-DSA-65	3	$6 \times 5 = 30$ hücre	3.309 B	1.952 B	4.032 B
ML-DSA-87	5 (en yüksek)	$8 \times 7 = 56$ hücre	4.627 B	2.592 B	4.896 B

Bu boyutlar tahmin değildir. Proje `fips204` adlı Rust kütüphanesini kullanır ve bir test her koşulda üretilen anahtar ve imzaların uzunluklarını FIPS 204 tablosuyla karşılaştırır. Uyuşmazsa yapı kırılır.

3.6 Hash fonksiyonları, BLAKE3, SHAKE-128 ve reddetme örnekleme

Hash fonksiyonu, herhangi uzunlukta bir girdiyi sabit uzunlukta bir çıktıya çeviren tek yönlü bir fonksiyondur. İyi bir hash üç özelliği sağlar: çıktıdan girdiye dönülemez, aynı çıktıyı veren iki girdi bulunamaz ve girdinin tek bir biti değişince çıktı tamamen değişir (çığ etkisi).

BLAKE3, projede genel amaçlı hash olarak kullanılır: p' tohumunun türetilmesi ve kısa tohumların üretilmesi.

SHAKE-128 bir XOF'tur (extendable-output function): sabit uzunlukta değil, istediğiniz kadar bayt üretebilen bir hash. Kafes matrisini doldurmak için tam olarak bu gerekir, çünkü 56 hücrenin her biri için taze rastgelelik lazımdır.

Reddetme örnekleme — neden gerekli

Matrisin her hücresi $[0, q)$ aralığında olmalı, yani 0 ile 8.380.416 arasında. SHAKE'den 3 baytlık (24 bit) bloklar okunur; 24 bitin aralığı $[0, 16.777.216)$ 'dir.

Ham değeri basitçe $\% q$ ile daraltmak yanlış olurdu. 16.777.216, 8.380.417'nin tam katı değildir; artan kısım yüzünden küçük değerler diğerlerinin iki katı olasılıkla çıkardı. Bu, matrisin dağılımını bozar ve güvenliği zayıflatır.

Çözüm reddetme örneklemesidir: okunan değer q 'dan küçükse kabul edilir, değilse atılır ve bir sonraki blok okunur. Kabul olasılığı $q / 2^{24} \approx \%49,9$ 'dur. FIPS 204 §7.3 (ExpandA) tam olarak bu yöntemi kullanır.

Denetimde çıkan bulgu. Bu kod eskiden Rust'ın DefaultHasher'ını (SipHash) kullanıyordu. İki sorun vardı: SipHash kriptografik değildir, ve çıktısının Rust sürümleri arasında aynı kalacağı garanti edilmez — yani aynı girdi başka bir derlemede başka bir matris üretebilirdi. Kanıtlar yeniden üretilebilir olmazdı. Şimdi SHAKE-128 + reddetme kullanılıyor.

3.7 Sıfır bilgi kanıtları ve ZK-STARK

Sıfır bilgi kanıtı, bir iddianın doğru olduğunu, iddianın kendisi dışında hiçbir bilgi sızdırmadan kanıtlama yöntemidir. Klasik örnek: bir labirentin içinden geçtiğinizi, hangi yoldan geçtiğinizi söylemeden kanıtlamak.

STARK (Scalable Transparent ARgument of Knowledge) bir kanıt sistemidir. İki özelliği önemlidir:

- Şeffaf: güvenilir bir kurulum töreni gerektirmez. (SNARK'lar genelde gerektirir; o kurulumdaki gizli veri sızarsa sahte kanıt üretilebilir.)
- Post-kuantum varsayımlara dayanır: güvenliği yalnızca hash fonksiyonlarına dayanır; eliptik eğrilere değil. Yani kuantum bilgisayar STARK'ı kırmaz.

STARK nasıl çalışır — üç adım

1) İz tablosu (execution trace). Kanıtlanacak hesaplama bir tabloya yazılır: her satır bir adım, her sütun bir değişken. Projede tablo 8 satır $\times$ 4 sütun'dur; sütunlar A_{commit} , s_1 , s_2 ve t 'dir.

2) AIR (Algebraic Intermediate Representation). Tablonun "doğru" sayılması için sağlaması gereken kurallar cebirsel denklemler olarak yazılır. Projede üç kısıt vardır:

```
result[0] = next[1] - (current[1] + 2)    # s1 her adımda +2
result[1] = next[2] - (current[2] + 3)    # s2 her adımda +3
```

```
result[2] = next[3] - (next[0] * next[1] + next[2])
# t = A·s1 + s2 (MLWE)
```

Her üç ifade de sıfır olmalıdır. Kanıtlayıcı, tablonun bu denklemleri her satırda sağladığını kanıtlar.

3) FRI (Fast Reed-Solomon IOP of Proximity). Tablo polinomlara çevrilir ve FRI, bu polinomların gerçekten düşük dereceli olduğunu — yani tablonun uydurulmadığını — rastgele noktalarda örnekleyerek doğrular. Doğrulayıcı tablonun tamamını görmez; birkaç noktaya bakarak ikna olur. Kanıtın özlü olmasının sebebi budur.

Projenin STARK parametreleri

Parametre	Değer	Anlamı
Alan	f128	128-bit asal alan (Winterfell kütüphanesi)
Güvenlik	80 bit (varsayımsal)	Sahte kanıt üretmek $\sim 2^{80}$ iş gerektirir. "Varsayımsal" (conjectured), kanıtlanmış değil tahmin edilen sınır demektir.
FRI sorgu sayısı	28	Doğrulayıcının baktığı rastgele nokta sayısı
Şişirme çarpanı	8	Polinomun kaç kat fazla noktada değerlendirildiği
Öğütme çarpanı	16	Kanıtlayıcıya eklenen ek iş yükü (grinding)
İz uzunluğu	8 satır	2'nin kuvveti olmak zorunda

80 bit, 96 değil. Belgeler bir dönem "96-bit" diyordu; kodda 80 yazıyordu. Artık tek bir sabitten (air.rs::STARK_SECURITY_BITS) okunuyor ve README'yi okuyan bir test ikisinin aynı kaldığını her koşulda doğruluyor.

3.8 Bu projede STARK tam olarak neyi kanıtlıyor

Bu, projenin en kritik ve en çok yanlış anlaşılan noktası. Açıkça yazalım.

Kanıtlanan	Kanıtlanmayan
p' tohumundan türetilmiş bir A matrisi ve s_1, s_2 dizileri için $t = A \cdot s_1 + s_2$ MLWE ilişkisinin 8 adım boyunca sağlandığı.	ML-DSA imzasının geçerli olduğu. İmza doğrulaması devre içinde ispatlanmıyor.
s_1 ve s_2 'nin tanımlı artış kuralına (+2, +3) uyduğu.	Gizli bir anahtarın bilindiği. Ortada gizlenen bir sır yok.

İmzalama ayrıca ve gerçekten yapılır: aynı p' tohumundan türetilen ξ ile $ML-DSA::keygen_from_seed(\xi)$ çağrılır, gerçek bir imza üretilir ve doğrulanır. Ama STARK ile imza arasındaki bağ ortak tohumdur, devre içi bir ispat değil.

"Sıfır bilgi" burada nominaldir. p' tohumu arayüzde ve payload'da yayınlanır. p' bilinince s_1, s_2 ve A matrisi herkes tarafından yeniden hesaplanabilir. Yani kanıt özlüdür (succinct — doğrulaması tablonun tamamını görmekten ucuz) ama hiçbir sır gizlemez. Bu bilinçli bir takastır: gizlilik yerine tekrarlanabilirlik seçildi, çünkü jürinin aynı girdiyle aynı kanıtı kendi makinesinde üretebilmesi daha değerli görüldü.

3.9 ERC-4337 — hesap soyutlama

Normal bir Ethereum cüzdanı (EOA, Externally Owned Account) programlanamaz: kuralları zincirin kendisi belirler ve yalnızca ECDSA imzası kabul edilir. Farklı bir imza şeması ya da "şu koşulda işlemi beklet" gibi bir kural ekleyemezsiniz.

ERC-4337 bu kısıtı kaldırır: cüzdanı bir akıllı sözleşme yapar. Böylece doğrulama mantığını siz yazarsınız. Projenin var olabilmesinin sebebi budur — uyarlanabilir zırh ancak programlanabilir bir cüzdanda mümkündür.

Bileşenler

Terim	Ne demek
UserOperation	Kullanıcının niyetini taşıyan yapı. Normal bir işlem değil; ayrı bir havuzda (mempool) dolaşır.
EntryPoint	Tüm ERC-4337 akışını yöneten tekil, denetlenmiş sözleşme. v0.7 sürümü CREATE2 ile tüm ağlarda aynı adrestedir: 0x0000000071727De22E5E9d8BAf0edAc6f37da032
Bundler	UserOperation'ları toplayıp EntryPoint'e gönderen aktör. Karşılığında ücret alır.
validateUserOp	Cüzdan sözleşmesinin doğrulama fonksiyonu. EntryPoint burayı çağırır; sonuç başarılıysa işlem yürütülür.
Paymaster	İşlemin gazını kullanıcı yerine ödeyen sözleşme. Kullanıcının hiç ETH'i olmadan işlem yapabilmesini sağlar.
ERC-7562	Doğrulama sırasında nelerin yasak olduğunu belirleyen kural seti. En önemlisi: doğrulama sırasında depolamaya yazmak yasaktır.

3.10 Yapay zekâ katmanı — Isolation Forest ve ONNX

Anomali tespiti, "normalden sapan" örnekleri bulma işidir. Burada zorluk şudur: elinizde etiketli saldırı verisi yoktur. Yeni bir drainer saldırısının nasıl görüneceğini önceden bilemezsiniz.

Isolation Forest tam bu durum için tasarlanmıştır. Mantığı ters yönden çalışır: veriyi rastgele bölmelerle parçalara ayırır ve bir noktanın tek başına kalması için kaç bölme gerektiğine bakar. Sıra dışı noktalar birkaç bölmede yalnız kalır; sıradan noktalar kalabalığın içinde gizlenir ve ayrılmaları uzun sürer.

Parametre	Değer
Ağaç sayısı	300
Contamination	0,03 (eğitim verisinin %3'ü anomali varsayılır)
Özellik sayısı	3 — İşlem_Sikligi, IP_Sapmasi, Gas_Sapmasi
Çalışma zamanı	ONNX Runtime (CPU)
Ölçülen çıkarım süresi	8,8-10,1 ms (5 koşu ortalaması, tek makine)

Üç özellik, on altı değil. Belgeler bir dönem "16 özellikli vektör" diyordu; koda üç sütun var. Düzeltildi. Eğitim verisi de kontrollü sentetiktir — canlı zincirden toplanmamıştır. Bu gizlenmiyor, kapsam sınırı olarak yazılıyor.

ONNX neden var

ONNX (Open Neural Network Exchange), eğitilmiş bir modeli eğitildiği kütüphaneden bağımsız, taşınabilir bir biçimde saklama standardıdır. Model scikit-learn ile eğitilir, ONNX'e dışa aktarılır ve çalışma anında ONNX Runtime ile koşturulur. Bir test, ONNX çıktısının orijinal scikit-learn çıktısıyla aynı kaldığını doğrular — dönüşümde bir şey bozulursa yakalanır.

Dinamik eşik $\tau(t)$ — sabit eşik neden yetmez

Sabit bir eşik ("skor 90'ı geçerse alarm") iki yönde de hatalıdır: ağ sakinken çok geç uyarır, ağ çalkantılıyken sürekli yanlış alarm verir. Projede eşik kayan pencere varyansından hesaplanır:

$$\begin{aligned}\tau(t) &= 60,0 + 0,15 \cdot \sigma^2(\text{gas_sapması}) + 0,08 \cdot \sigma^2(\text{işlem_sıklığı}) \\ \tau(t) &= \text{clamp}(\tau(t), 55,0, 90,0)\end{aligned}$$

- Pencere son 50 gözlemi tutar.
- Varyans Bessel düzeltmesiyle (ddof=1) hesaplanır — pencere, tüm nüfusun değil bir örneklemin özetidir.
- Pencerede 5'ten az gözlem varsa soğuk başlangıç değeri 75,0 kullanılır.
- Sonuç [55,0 – 90,0] aralığına sıkıştırılır; eşik ne tespit edilemez ne de her şeyi anomali sayan bir değere saplanır.

3.11 Hareketli Hedef Savunması (MTD)

Moving Target Defense, saldırganın keşif yaptığı yüzeyi sürekli değiştirme fikridir. Sabit bir hedefi incelemek için zamanı olan saldırgan avantajlıdır; hedef değişirse bu avantaj kaybolur.

Projede bu, ρ' rotasyonu ile yapılır. Her rotasyonda yeni bir ρ' tohumu türetilir ve A matrisinin tamamı yeniden genişletilir. ρ'' 'nin tek bir biti değişirse 56 hücrenin tamamı değişir — bir test bunu 56/56 olarak doğrular.

Rotasyon kafes probleminin zorluğunu artırmaz. MLWE'nin zorluğu parametrelerden gelir, tohumdan değil. Rotasyonun sağladığı şey ileri güvenliktir (forward secrecy): saldırganın eski tohuma dair topladığı bilgi geçersiz olur. Bu ayrım arayüzdeki sınırlar panelinde de yazılıdır.

4. Mimari — dört katman

Sistem dört katmandan oluşur. Her katman ayrı bir dilde yazılmıştır ve bu tesadüf değildir: her dil, kendi katmanının gerektirdiği işte en iyisidir.

Katman	Dil / Teknoloji	Sorumluluğu	Neden bu
1. Yapay zekâ	Python · ONNX · FastAPI	Davranışı puanlar, dinamik eşiği hesaplar, kararı verir	Makine öğrenimi ekosistemi burada
2. Kriptografi	Rust · fips204	Gerçek ML-DSA anahtar üretimi, imzalama, doğrulama	Bellek güvenliği yüksek başlangıç için standarta uyum
3. Kanıt	Rust · Winterfell	İz tablosu, AIR kısıtları, STARK kanıtı	Winterfell cümle Rust STARK kütüphanesi
4. Zincir	Solidity · ERC-4337	Kararı uygular, guardian imzasını doğrular, gazı sponsorlar	EVM'in tek kuralı

4.1 Veri akışı

```

Kullanıcı / senaryo
  | 3 özellik: İşlem sıklığı, IP sapması, Gas sapması
  ▼
[1] Python API — ONNX Isolation Forest → risk skoru
    |          dinamik eşik  $\tau(t)$  ile karşılaştırılır
    | risk <  $\tau$  → NORMAL: kanıt üretilmez, taban kademe
    | risk ≥  $\tau$  → PANİK: aşım göre kademe seçilir
    ▼ (6 argüman: risk, tau, taban, userOpHash, epoch, run-id)
[2] Rust ikilisi
    | hashing::derive_rho_prime →  $\rho'$  (32 bayt, BLAKE3)
    | trace::from_rho_prime → kafes + kısa tohumlar
    | pqc::sign_and_verify → GERÇEK ML-DSA imzası
    | [3] Winterfell → STARK kanıtı
    ▼ proof_payload.json
[1] Python API — ölçümleri yanıtı taşır → arayüz
    ▼ guardian ECDSA attestation imzası
[4] Akıllı sözleşme (ERC-4337)
    | kanıt uzunluğu kontrolü (≥ 3000 bayt)
    | guardian imzası ecrecover ile doğrulanır
    | zırh kademesi güncellenir (yalnızca YUKARI)

```

4.2 Kararı veren tek kural

Zırh kademesini seçen mantık tek bir fonksiyondadır ve hem Rust hem Python aynı kuralı uygular:

```

risk <  $\tau$           → taban kademe, kanıt YOK, durum = NORMAL
risk ≥  $\tau$           → aşım = risk -  $\tau$ 
    aşım ≥ 15.0      → ML-DSA-87
    aşım ≥ 5.0       → ML-DSA-65
    aksi             → ML-DSA-44
risk veya  $\tau$  = NaN → ML-DSA-87 + panik (sessizce normale düşmez)

```

```
her durumda: seçilen = max(seçilen, taban) ← tek yönlü tırmanma
```

Neden iki dilde aynı kural var. Eskiden Rust sabit bir 90,0 eşiği kullanıyordu, Python ise dinamik $\tau(t)$. İkisi farklı karar verebiliyordu. Şimdi tek kural var ve 13 sınır vektörüyle bir eşitlik testi, Python'un kararıyla gerçek Rust ikilisinin kararının aynı olduğunu doğruluyor.

4.3 Tek yönlü tırmanma

Zırh kademesi asla düşürülemez — ne zincir dışında ne de zincir üstünde. Bir saldırgan EntryPoint üzerinden çağrı yapıp kademeyi ML-DSA-87'den 44'e indirmeye çalışırsa reddedilir.

Bu, iki fuzz testiyle korunur: rastgele üretilmiş binlerce girdi dizisi için kademenin monoton arttığı ve asla tabanın altına inmediği sınanır.

5. Katmanlar ayrıntılı

5.1 Yapay zekâ katmanı (Python)

Sorumlulukları

- 3 özellikli vektörü alıp ONNX modeliyle ham anomali skoru üretmek.
- Skoru Platt ölçekleme ile $[0, 100]$ aralığına kalibre etmek.
- Kayan pencere varyansından $\tau(t)$ hesaplamak.
- Karar panikse Rust ikilisini altı argümanla çağırmak.
- Üretilen payload'ı okuyup arayüze taşımak.
- Guardian attestation imzasını üretmek.

DoS koruması — kuyruk kapasitesi

STARK kanıtı üretmek CPU yoğun bir iştir. Saldırgan arka arkaya yüksek riskli istek göndererek sunucuyu kilitleyebilir. Bu yüzden kanıt üretimi sınırlı bir `asyncio.Queue`'ya alınır; kuyruk dolarsa HTTP 429 döner.

Kapasite sabit değildir ve bu önemlidir. Eski sürüm `maxsize=50` yazıyordu ve bu sayının hiçbir dayanağı yoktu. Daha kötüsü: 2 çekirdekli bir makinede 50 eşzamanlı kanıt açmak korumayı işlevsiz kılar — kuyruk hiç dolmaz ama makine çöker. Koruduğunu iddia ettiği senaryoda devreye girmeyen bir savunma, savunma değildir. Kapasite artık çekirdek sayısı ve boş bellekten türetilir; gerekçe `/api/health` üzerinden metin olarak yayınlanır.

5.2 Kriptografi ve kanıt katmanı (Rust)

ρ' tohumu nasıl türetilir

```
 $\rho' = \text{BLAKE3}(\text{ALAN\_ETİKETİ} \parallel \text{risk\_bitleri} \parallel \text{epoch\_ns} \parallel \text{len}(\text{userOpHash}) \parallel \text{userOpHash} \parallel \text{entropi\_bayrağı})$ 
```

- Alan etiketi, aynı hash'in başka amaçlarla üretilen özetleriyle çakışmayı önler.
- Uzunluk ön-eki şart: ("ab" $\parallel$ "") ile ("a" $\parallel$ "b") aynı özete gitmemeli.
- Entropi bayrağı 0 ise koşu deterministiktir. Taze entropi isteyen `--fresh-entropy` ile açıkça verir ve bu payload'da işaretlenir.

Determinizm neden bilinçli. Eski uygulama `process::id()` karıştırıyordu; her koşuda farklı olduğu için kanıt yeniden üretilebilir değildi — jüri aynı sonucu alamazdı. Kaldırıldı.

ρ' nereye gidiyor

Hedef	Nasıl
Gerçek ML-DSA anahtarı	$\xi = \text{BLAKE3}(\text{alan} \parallel \rho') \rightarrow \text{KG}::\text{keygen_from_seed}(\xi)$. Bu gerçek bir FIPS 204 anahtar üretimidir, benzetim değil.
STARK iz tablosu	Kafes matrisi SHAKE-128 + reddetme örneklemeyle genişletilir (FIPS 204 §7.3 ExpandA ile aynı yordam).

Gösterilen iz = kanıtlanan iz

Bir dönem arayüzde gösterilen tablo ile STARK'ın kanıtladığı tablo farklıydı: biri u128 aritmetiği, diğeri alan aritmetiği kullanıyordu. Sahnede jüriye gösterilen şey kanıtlanan şey değildi. Artık Winterfell tablosu, gösterilen tablodan kopyalanır — yeniden hesaplanmaz — ve bir test ikisinin aynı olduğunu doğrular.

5.3 Zincir katmanı (Solidity)

Sözleşme	Görevi
QAdaptiveAccount	ERC-4337 akıllı cüzdan. Zırh kademesini tutar, guardian imzasını doğrular, yüksek değerli transferlerde zaman kilidi uygular.
QAdaptivePaymaster	Gaz sponsorluğu. Dört bağımsız kapıdan geçmeyen işlem sponsorlanmaz.
QAdaptiveAICore	Zincir üstü risk oracle'ı. Zincir dışı guardian buraya risk yazar; cüzdan buradan okur.

Zincirde ne oluyor, ne olmuyor

Sözleşme STARK kanıtını doğrulamaz. Yaptığı iki şey vardır:

- Kanıtın uzunluğunu kontrol eder (`MIN_STARK_PROOF_BYTES = 3000`). Panik modunda kanıt zorunludur ve bu uzunluğu sağlamalıdır.
- Guardian'ın ECDSA attestation imzasını ecrecover ile doğrular. Bu imza, zincir dışı sistemin kararını zincire taşıyan bağıdır.

Ölçülen maliyet: `validateUserOp` için 26.449 - 147.510 gaz (medyan 74.155), `forge test --gas-report` ile ölçüldü.

Oracle bayatlarsa ne olur — tasarımın kalbi

Bir oracle'ın susması, hiç konuşlandırılmamış olmasından daha tehlikelidir: cüzdan son değere güvenmeye devam eder. İki yön de kötüdür:

Yaklaşım	Sonuç
fail-open — bayatken risk = 0 raporla	Updater'ı susturabilen saldırgan zırhı en zayıf kademeye düşürür. Saldırımı ödüllendirir.
fail-closed — bayatken panik = true raporla	Her işlem ≥ 3.000 baytlık kanıt ister. Zincir dışı yığın çöktüğü için bayatlamıştır; o kanıtı kimse üretemez ve cüzdan kilitlenir.
seçilen — azami risk, panik KAPALI	Zırh en güçlü kademeye çıkar (savunmacı, kullanıcıya zincirde ek maliyeti yok) ama cüzdan kullanılabilir kalır.

Yeni konuşlandırılan oracle bilerek bayat başlar: henüz hiçbir şey ölçmemiştir, risk 0 raporlamak en tehlikeli yönde yalan olurdu. `isStale()` herkese açıktır, böylece arayüz bozulmuş durumu gizlemek yerine gösterebilir.

6. Uçtan uca bir koşu

Bu bölüm, yüksek riskli bir işlemin sistemden nasıl geçtiğini adım adım izler. Aşağıdaki süreler gerçek bir koşudan alınmıştır; makinenize göre değişir.

6.1 Senaryo

İşlem sıklığı : 2.0 tx/s
IP sapması : 0.95
Gas sapması : 15.5

Bu profil bir «drainer» saldırısına benzer: alışılmadık IP coğrafyası + aşırı yüksek gas kullanımı.

6.2 On bir aşama

Boru hattının her aşaması ayrı ayrı ölçülür. Arayüzdeki şeritte gördüğünüz süreler bunlardır:

#	Aşama	Ne yapar	Örnek süre
1	ONNX çıkarım	3 özellik → Isolation Forest → kalibre risk skoru	13,13 ms
2	zırh kararı	risk ile $\tau(t)$ karşılaştırılır, kademe seçilir	0,01 ms
3	ρ' üretimi	BLAKE3 ile 32 baytlık tohum	0,01 ms
4	kafes genişletme	SHAKE-128 + reddetme → $8 \times 7 = 56$ hücre	0,12 ms
5	ML-DSA keygen	ξ 'den gerçek anahtar çifti (pk 2592 B, sk 4896 B)	0,58 ms
6	imzalama	Gerçek ML-DSA imzası (4.627 B)	0,55 ms
7	doğrulama	İmza geçerli mi — evet	0,35 ms
8	kurcalama testi	Mesaj bozulursa imza reddediliyor mu — evet	0,35 ms
9	iz tablosu	8 satır $\times$ 4 sütun MLWE izi	0,05 ms
10	STARK prover	Winterfell kanıtı üretir (~4.009 bayt)	6,81 ms
11	yerel doğrulama	Kanıt yerelde doğrulanır, 80 bit konjektürel	0,15 ms

Neden 11, 12 değil. Listede bir dönem payload_yazma adlı 12. bir aşama vardı ve her zaman 0,00 ms gösteriyordu: prover kendi dosya yazımını ölçemez. Şeridin başlığı «HER SÜRE ÖLÇÜLDÜ» iken orada ölçülmemiş bir sıfır durmasına izin verilemezdi. Hem Rust hem Python tarafından kaldırıldı ve iki ayrı test geri gelmesini engelliyor.

6.3 Kurcalama testi neden canlı hatta koşuyor

8. aşama dikkat çekicidir. "Kurcalanmış mesaj reddediliyor" iddiası eskiden yalnızca bir birim testindeydi. Artık her koşuda, sahnenin ortasında çalışır: imzalanan mesaj bozulur ve doğrulayıcının onu reddettiği görülür. Reddetmezse koşu durur.

Bunun anlamı şudur: jüri "imzanız gerçekten çalışıyor mu?" diye sorduğunda, cevap bir teste atıf değil — ekranda o anda yanan bir rozettir.

6.4 Calldata tasarrufu — dikkatli okunması gereken sayı

Sistem 50 işlemlik bir parti için tek bir kanıt üretir. Karşılaştırma şöyledir:

```
İşlem başına ML-DSA-87 imzası : 50 × 4.627 B = 231.350 B
Tek ZK-STARK kanıtı          : ~4.009 B
                             → yaklaşık %98,3 tasarruf

Karşılaştırma: 50 ECDSA imzası : 50 × 65 B = 3.250 B
                             → ECDSA DAHA KÜÇÜK
```

Taban ML-DSA imzasıdır, ECDSA değil. 50 ECDSA imzası tek bir STARK kanıtından küçüktür; yani ECDSA calldata boyutunda bizi yener. calldata.beats_ecdsa alanı bu yüzden false döner ve arayüz bunu gizlemez. Takas edilen şey boyut değil, post-kuantum güvenliğidir — ECDSA'nın Shor algoritması altında sağlayamadığı tek şey.

7. Arayüz

Arayüz tek bir HTML dosyasıdır ve hiçbir dış kaynak kullanmaz — ne Tailwind, ne Chart.js, ne Google Fonts. Sebebi pratiktir: sahnede internet olmayabilir. Gömülü CSS, vanilla JavaScript, satır içi SVG ve sistem yazı tipleriyle yazılmıştır. Bir test bu kuralı zorlar.

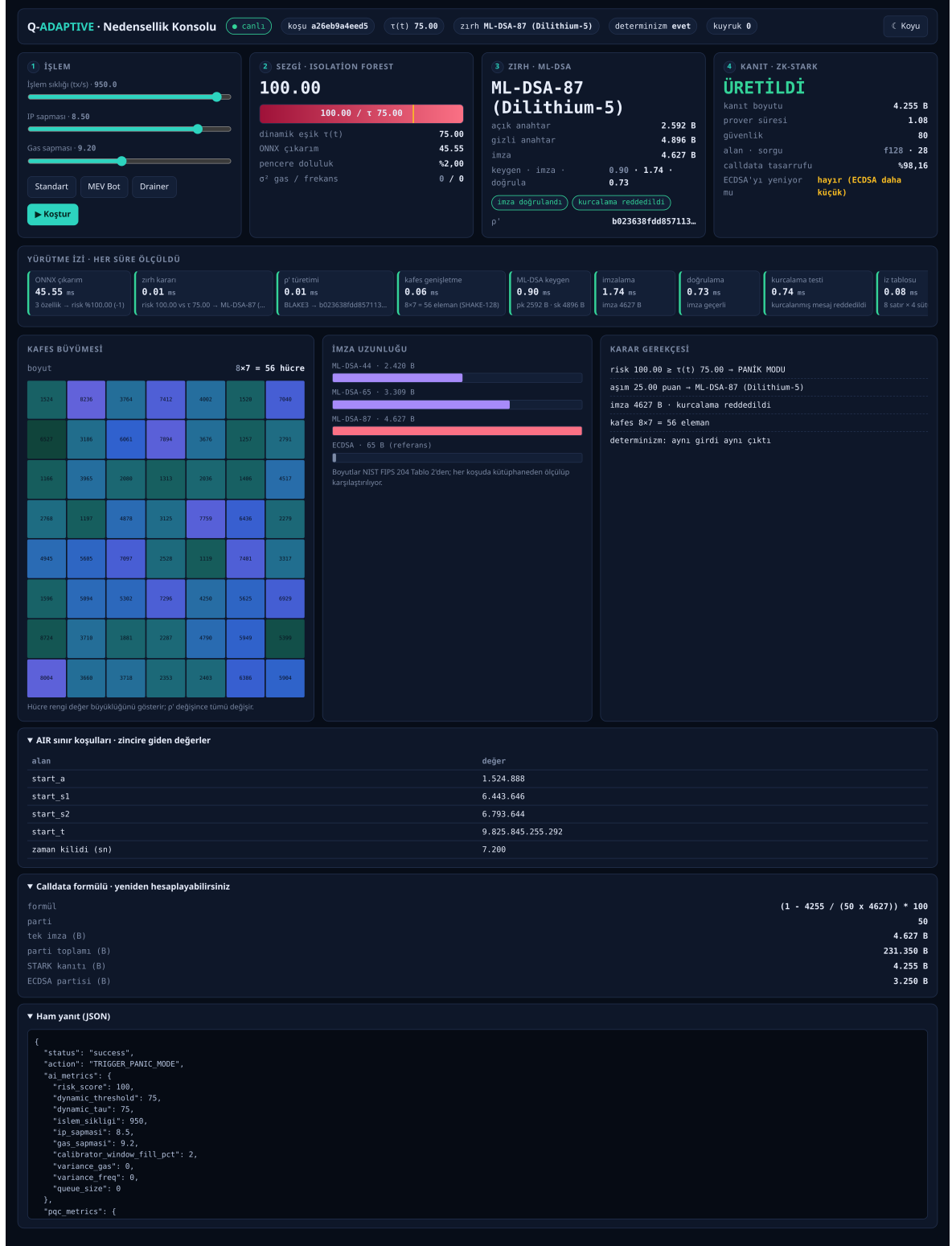
7.1 İki temel kural

Kural	Nasıl uygulanıyor
Hiçbir sayı uydurulmaz	Her bağlı alan data-bind="yol.alan" ile işaretli. Bir test bu yolların API şemasında gerçekten var olduğunu doğruluyor — 38 alan. Arayüz var olmayan bir alana bağlanırsa CI kırılır.
Sunucu yoksa hiçbir şey gösterilmez	Bağlantı koparsa kırmızı bant çıkar ve 38 alanın 38'i — olur. Eski arayüz sessizce örnek veriye düşüyordu; bu, denetimde kapatılan «bayat kanıt geri dönüşü» bulgusunun arayüz hâliydi.

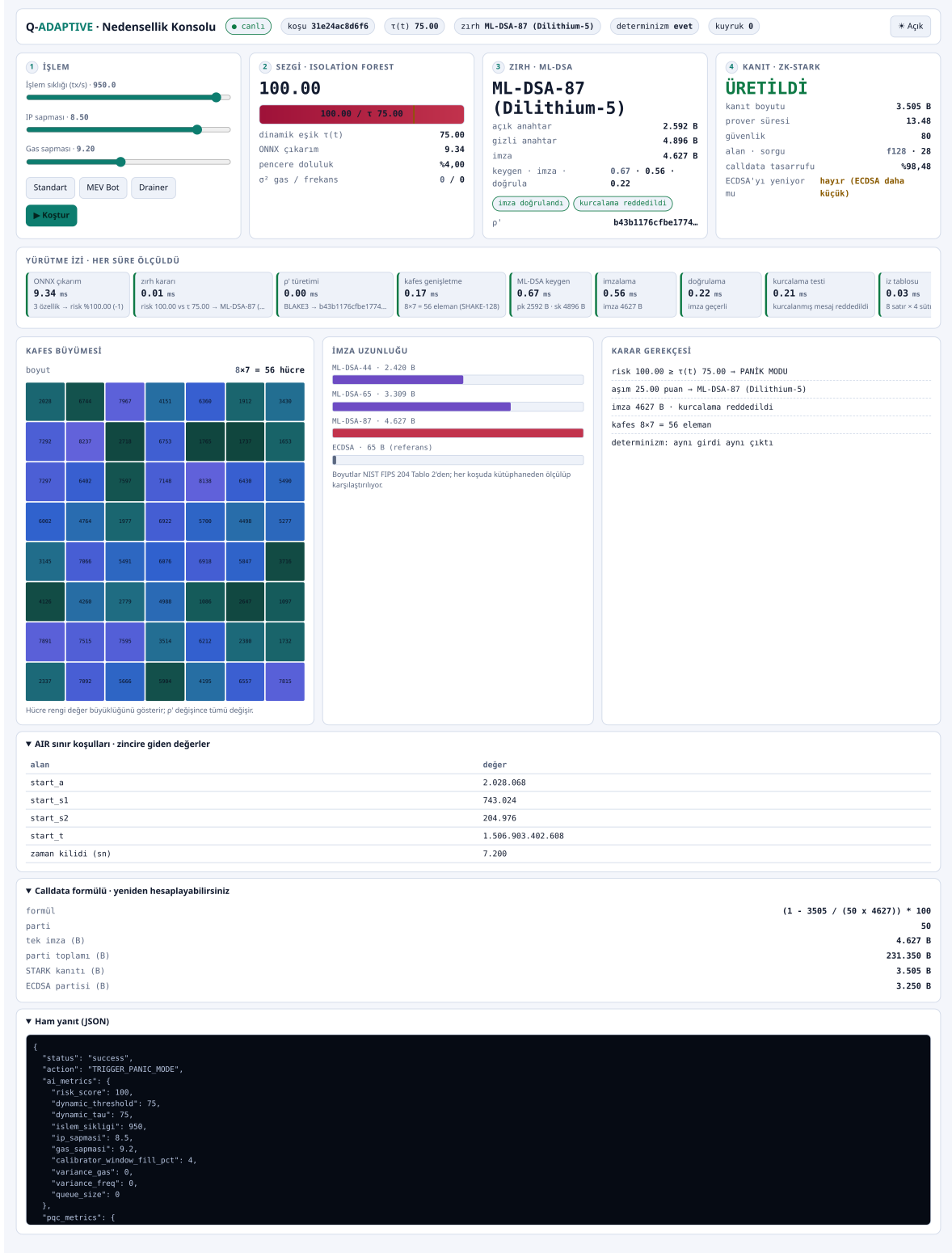
7.2 Tema

Varsayılan tema koyudur: sunum karartılmış bir salonda projeksiyonla yapılacak. Aydınlık odada incelemek ve ekran görüntüsü almak için açık tema vardır; üst şeritteki ☾/☀ düğmesi veya T tuşuyla geçilir ve tercih saklanır.

7.3 Tam ekran



Görüntü 1 — Konsolun tamamı (koyu tema). Ekran soldan sağa, yukarıdan aşağıya bir nedensellik sırası izler: işlem → sezgi → zırh → kanıt, ardından yürütme izi, sonra kafes ve karar gerekçesi. Sekme yoktur; bir koşulunun tüm parçaları aynı ekranda ve neden-sonuç sırasındadır.



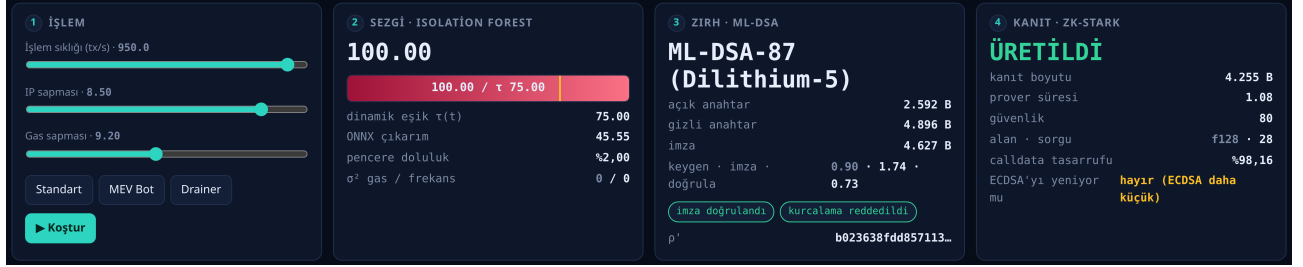
Görüntü 2 — Aynı konsol, açık tema. Vurgu renkleri beyaz zeminde WCAG AA kontrastını sağlayacak biçimde ayrı seçilmiştir; koyu tema için ayarlanmış tonlar doğrudan kullanılsaydı okunabilirlik düşerdi.

7.4 Üst şerit



Görüntü 3 — Durum şeridi. Soldan sağa: bağlantı durumu, koşu kimliği (run_id), dinamik eşik $\tau(t)$, seçilen zırh kademesi, koşunun deterministik olup olmadığı ve kuyruk derinliği. Bağlantı koparsa bu şerit kırmızıya döner ve ekrandaki tüm alanlar temizlenir.

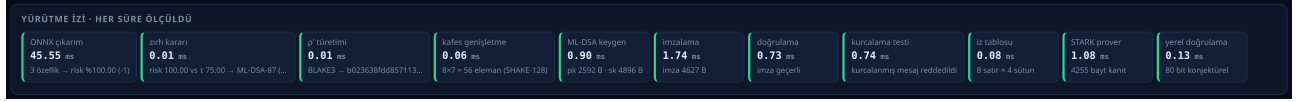
7.5 Nedensellik: işlem → sezgi → zırh → kanıt



Görüntü 4 — Dört kart, tek satırda projenin tüm savı. Kart 1 üç girdi özelliğini ve senaryo düğmelerini tutar. Kart 2 Isolation Forest skorunu $\tau(t)$ ile karşılaştırır ve ölçülen ONNX süresini gösterir. Kart 3 bu karşılaştırmanın seçtiği zırh kademesini, gerçek FIPS 204 anahtar/imza boyutlarını ve canlı kurcalama kontrolünü gösterir. Kart 4 STARK sonucunu taşır: kanıt boyutu, prover süresi, güvenlik biti ve calldata tasarrufu.

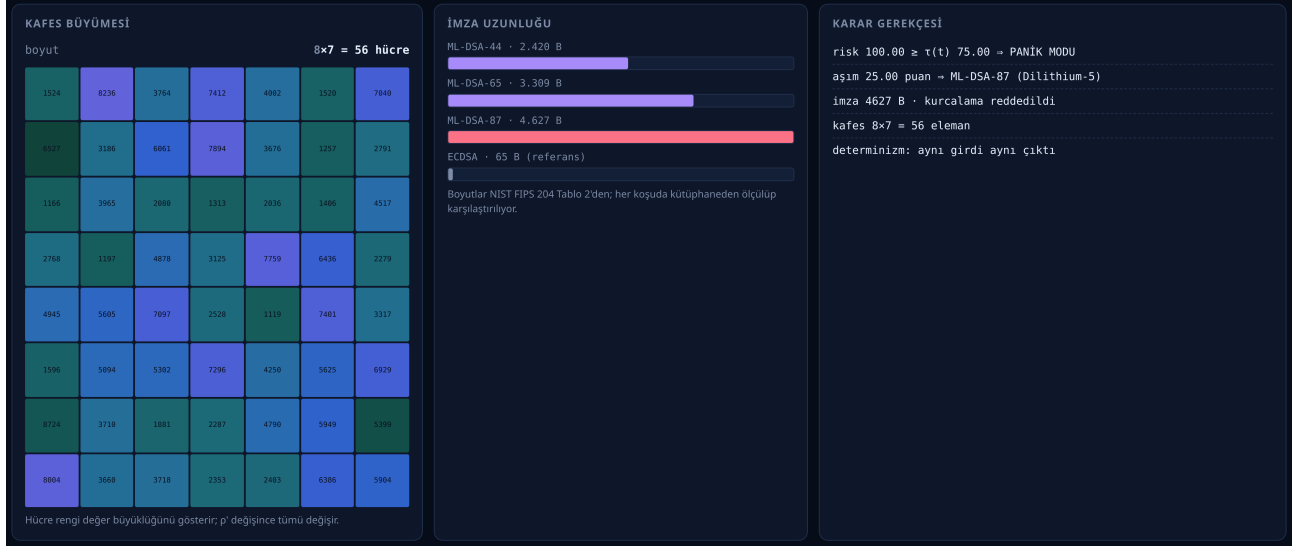
Kart 4'ün son satırına dikkat: «ECDSA'yı yeniyor mu → hayır (ECDSA daha küçük)». Bu satır bilerek oradadır. Sistem kendi aleyhine olan ölçümü de gösterir; gizlemez.

7.6 Yürütme izi



Görüntü 5 — On bir aşama, her birinin süresi ayrı ölçülmüş. ONNX çıkarımından p' türetimine, kafes genişletmeden ML-DSA anahtar üretimi/imzalama/doğrulamaya, canlı kurcalama testinden iz tablosuna, STARK prover'dan yerel doğrulamaya. Her kutuda üstte aşama adı, ortada ölçülen süre, altta o aşamanın ne yaptığına dair tek satır kanıt (örneğin «pk 2592 B · sk 4896 B» ya da «kurcalanmış mesaj reddedildi»).

7.7 Kafes büyümesi ve karar gerekçesi



Görüntü 6 — Üç sütun. Solda p'den SHAKE-128 ve reddetme örneklemeyle genişletilmiş gerçek kafes matrisi; ML-DSA-87 kademesinde 8×7 = 56 hücre, hücre rengi değerın büyüklüğünü gösterir. p'nün tek bir biti değişirse 56 hücrenin tamamı değişir. Ortada üç ML-DSA imza uzunluğunun ECDSA referans çubuğuyla karşılaştırması — değerler FIPS tablosundan değil, o koşuda ölçülmüş kütüphane çıktısından gelir. Sağda bu koşunun neden böyle bittiğinin, kodun değerlendirdiği sırayla yazılmış gerekçesi.

7.8 Arayüzde olmayan şey: sınırlar paneli

Konsolda bir dönem «İddia Etmediklerimiz» paneli ve altı adımlık bir Sunum Modu vardı. İkisi de kaldırıldı; arayüz tek bir çalışma ekranı olarak sadeleştirildi.

Sınırlar ortadan kalkmadı, yer değiştirdi. Bu belgenin 9. bölümü ve jüri kitapçığının 4. bölümü onları ayrıntılı olarak sayıyor. README'de de bir tablo hâlinde duruyor ve bir test, maddelerden biri silinirse yapıyı kırıyor.

Bunun sunuma etkisi: sınırlar artık ekranda kendiliğinden görünmüyor, yani sözlü olarak söylenmeleri gerekiyor. Jüri «burada sıfır bilgi olan ne?» diye sorduğunda işaret edilecek bir panel yok; cevabın hazır olması şart. Jüri kitapçığı tam olarak bunun için var.

Arayüzde kalan tek klavye kısayolu T (tema) ve Enter (koşuyu tetikle).

8. Neler yapabiliyor

Aşağıdaki her satır çalıştırılarak doğrulanmıştır ve yanında onu koruyan test vardır.

Yetenek	Kanıtı	Koruyan test
Gerçek ML-DSA anahtarı üretip imza atıyor ve doğruluyor	fips204 kütüphanesi; boyutlar FIPS 204 tablosuyla karşılaştırılıyor	pqc::tests::gercek_imza_ve_dogrulama, standart_boyutlari_uyusuyor
Kurcalanmış mesajı reddediyor	Canlı hatta her koşuda sınanıyor, sonuç ekranda rozet	pqc::tests::kurcalanan_mesaj_reddediliyor
Zırh risk ile birlikte gerçekten büyüyor	İmza 2.420 → 3.309 → 4.627 bayta çıkıyor	pqc::tests::imza_zirhla_birlikte_buyuyor
AI kararı kripto katmanına geçiyor	Altı argüman ikiliye aktarılıyor; kafes 16 → 30 → 56 hücre	pipeline::tests::otonomi_koprusu_riski_kripto_katmanina_tasiyor
Rust ve Python aynı eşik kuralını uyguluyor	13 sınır vektörü, gerçek ikili çalıştırılarak	ArmorPolicyParityTest
Aynı girdi aynı kanıt veriyor (determinizm)	ρ , anahtar ve imza birebir aynı	hashing::tests::rho_prime_tam_deterministik, DeterminismParityTest
Gösterilen iz = kanıtlanan iz	Winterfell tablosu kopyalanıyor, yeniden hesaplanmıyor	pipeline::tests::gosterilen_iz_kanitlanan_izle_ayni
Python imzası gerçek sözleşmede doğrulanıyor	Saf Python Keccak-256 + secp256k1 → Solidity ecrecover	GuardianAttestationTest
Paymaster boşaltılamıyor	Dört bağımsız kapı; sömürü senaryosunun kendisi bir test	test_BULGU5_saldirgan_sozlesmesi_sponsorlu_k_alamiyor
Risk skoru kullanıcıdan okunamıyor	Oracle ya da guardian imzası; gönderenin yazdığı alan karara girmiyor	test_BULGU6_iddia_edilen_sifir_risk_ai_kapisini_gecemiyor
Zırh asla düşürülemiyor	Zincir üstünde tek yönlü tırmanma	testFuzz_E4_zirh_monoton_artiyor
Gerçek EntryPoint v0.7 ile çalışıyor	Ethereum mainnet fork'una karşı 4/4 test	EntryPointFork.t.sol
Oracle susarsa zırh düşmüyor	Bayat oracle azami risk + panik kapalı raporluyor	test_updateri_susturmak_zirhi_dusurmuyor

9. Neler yapamıyor ve neden

Bu bölüm bilinçli olarak ayrıntılıdır. Bir sınırı gizlemek onu ortadan kaldırmaz; yalnızca başkasının bulmasını bekler.

9.1 Mimari sınırlar — tasarım gereği

Sınır	Sebebi	Düzeltililebilir mi
STARK, ML-DSA doğrulamasını devre içinde ispatlamıyor	AIR üç kısıt uygular: s_1 ve s_2 'nin ilerleyişi ve $t = A \cdot s_1 + s_2$. Bu, ML-DSA imza doğrulama devresi değil, ondan esinlenmiş bir MLWE ilişkisidir.	Pratikte hayır. Tam ML-DSA doğrulamasını bir STARK devresine kodlamak aylar süren, araştırma düzeyinde bir iştir.
Kanıt zincirde doğrulanmıyor	Sözleşme yalnızca uzunluk kontrolü + ECDSA attestation yapar.	Hayır. Solidity'de STARK doğrulayıcı yazmak devasa bir iştir ve mimarinin amacı zaten bundan kaçınmaktır.
«Sıfır bilgi» nominal — gizlenen sır yok	p' yayınlanıyor; s_1 , s_2 ve A ondan yeniden hesaplanabilir.	Teknik olarak evet, ama istenmez. p' 'yi gizlemek determinizmi ve jürinin kanıtı yeniden üretebilmesini bozardı.
AI kuantum saldırısı tespit etmiyor	Model davranışsal anomali görür; kuantum saldırısının zincirde bıraktığı bir iz yoktur.	Hayır. Bu, modelin değil problemin doğası.
Rotasyon kafes problemini zorlaştırmıyor	MLWE'nin zorluğu parametrelerden gelir, tohumdan değil.	Uygulanamaz. Rotasyonun amacı ileri güvenlik.

9.2 Eksikler — yapılabilir ama yapılmadı

Eksik	Bugünkü durum	Değerlendirme
Hiçbir ağa konuşlandırılmadı	Konuşlandırma betiği yazıldı, testlerle korunuyor ve yerelde anvil'e karşı uçtan uca koştu. Genel bir ağa gönderilmedi.	Yapılmalı. Fonlu bir anahtar ve RPC ile birkaç saat. «Fork testi geçti» ile «testnet'te canlı» arasında büyük fark var.
Bağımsız güvenlik denetimi yok	Slither ve solhint CI'da koşuyor; insan denetimi yapılmadı.	Yarışma için gerekli değil. Üretim için şart.
Guardian anahtarı bellekte	attestation.py anahtarı süreç belleğinde tutuyor ve sabit-zamanlı değil.	Üretim şartı, prototip şartı değil. HSM/KMS'e taşınmalı.
Account dal kapsamı %80,28	Kapsanmayan dallar guardian imza kurtarma savunma kolları (bozuk v, sonsuzdaki nokta).	Yapılabilir, 1–2 saat. Orta değerli.
Eğitim verisi sentetik	Kontrollü üretilmiş veri; canlı zincirden toplanmadı.	Gizlenmiyor. Gerçek veriyle eğitim günler alır ve yarışma kapsamını aşar.
Yüzdelik gecikmeler ölçülmedi	Ortalama ölçülüyor; p50/p95/p99 toplanmıyor.	Kolay ama düşük değerli. Belgeler artık «ölçülmedi» diyor.

9.3 Söylenemeyecek cümleler

Bunlar sahnede kullanılmamalıdır, çünkü doğru değildirler:

- «STARK, ML-DSA doğrulamasını ispatlıyor» → ispatlamıyor.
- «Kanit zincirde doğrulanıyor» → doğrulanmıyor, uzunluğu kontrol ediliyor.
- «Mainnet'e konuşlandırıldı» → hiçbir ağa konuşlandırılmadı.
- «ECDSA'dan daha az calldata» → 50 ECDSA imzası bizden küçük.
- «Bağımsız denetimden geçti» → geçmedi.
- «%100 test kapsamı» → dal kapsamı %82,55.
- «Sertifikalı metrikler» → hiçbir kurum sertifikalandırmadı; kendi ölçümlerimiz.

10. Test ve doğrulama

Projenin işleyiş kuralı şudur:

Bir bulgu, ancak geri gelmesini engelleyen ve gerçekten çalıştırılmış bir testi varsa kapalıdır. Yorumda «düzeltildi» yazmak kapatmaz.

10.1 Sayılar

Katman	Komut	Sonuç
Rust (ZK + PQC)	cargo test	61 geçti
Solidity	forge test	142 geçti, 4 atlandı (fork)
Katman eşitliği	python3 test_layer_parity.py	9 geçti
Attestation kriptosu	python3 test_attestation.py	18 geçti
API ↔ arayüz sözleşmesi	python3 test_api_contract.py	9 geçti
Toplam		244 otomatik test

Solidity testlerinin 12'si fuzz/değişmez testidir; her biri 512 rastgele koşu yapar.

10.2 Ölçülen kapsam

Sözleşme	Satır	Dal	Fonksiyon
QAdaptiveAICore	%100,00 (46/46)	%95,24 (20/21)	%100,00 (11/11)
QAdaptivePaymaster	%100,00 (104/104)	%97,14 (34/35)	%100,00 (17/17)
QAdaptiveAccount	%94,15 (177/188)	%80,28 (57/71)	%96,00 (24/25)
Toplam	%96,10	%82,55	%95,31

Dal kapsamı %100 değildir ve öyle olduğu iddia edilmez.

10.3 CI hattı — altı iş

İş	Ne yapar
Python · ONNX & API	ONNX parity, attestation, sözleşme testleri
Rust · STARK & Clippy	cargo test, clippy -D warnings, fmt --check
Solidity · Foundry	forge test, coverage, gas report, deploy build
Regresyon Koruması	Kapatılan bulgular grep ile geri gelmesin
Solidity · Lint & Slither	solhint + Slither zafiyet dedektörleri
Repo · Hijyen	TruffleHog sır taraması, dosya hijyeni

10.4 Alışılmadık korumalar

Koruma	Neyi engelliyor
README'yi okuyan test	Belgedeki güvenlik biti ile koddaki sabitin ayrışması
Arayüzü okuyan test	Arayüzün API'de var olmayan bir alana bağlanması (38 alan)
Sınırlar panelini okuyan test	Bir sınırın panelden sessizce silinmesi
Belge kod bloklarını üreten kapı	docs/ içindeki kod alıntılarının kaynaktan sapması (23 blok)
Grep tabanlı regresyon kapıları	DefaultHasher, sabit 90,0 eşiği, uydurma 4608 tabanı, process::id(), gas: 2300 stipend'i

11. Denetim hikâyesi — çıkarılan dersler

Bu bölüm projenin teknik değil yöntemsel kısmıdır ve belki de en öğreticisidir. Proje bir denetimden geçti; 21 bulgu çıktı ve hepsi kapatıldı. Ama asıl değerli olan, bulguların ortak deseni idi.

11.1 Kurucu bulgu

Belgeler «ML-DSA kullanıyoruz» diyordu. Kodda tek satır ML-DSA yoktu — depoda hiçbir post-kuantum bağımlılığı bile bulunmuyordu.

Bu, tek bir hata değil bir hata sınıfıydı: anlatının koddan kopması. Aynı desen sonra dört ayrı yerde daha bulundu.

11.2 Aynı hatanın dört görünümü

Nerede	Ne oluyordu
Kodda	Belgeler ML-DSA diyordu, kod yoktu. Zırh geçişi JSON'a yazılan bir metinden ibaretti.
Arayüzde	API'ye ulaşılamazsa sessizce örnek veriye düşüyordu (proof_size_kb: 3.85, risk_score: 98.52). Sunucu çökse bile ekran dolu görünüyordu.
CI'da	Dört ayrı «yeşil ama hiçbir şey yapmayan» kontrol: geçersiz YAML, hiçbir şey taramayan TruffleHog, sessizce çöken Slither, pipefail'siz boru hattı.
Belgelerde	docs/ altındaki «Kod İncelemesi» bölümleri kaynak dosyaların tamamını alınılıyordu, ama alıntılar donmuştu: DefaultHasher koddan silindikten sonra belgede 21 yerde duruyordu.

11.3 En tehlikeli desen: yeşil ama boş kontrol

Bu oturumda dört kez aynı tuzağa düşüldü ve her seferinde ders aynıydı:

Koştuğu iddia edilen ama hiçbir şey yapmayan bir kontrol, hiç kontrol olmamasından daha tehlikelidir — çünkü sahte bir güvence verir. "CI yeşil" demek, CI'ın bir şey kontrol ettiği anlamına gelmez.

- CI geçersiz YAML yüzünden hiç koştumuyordu.
- TruffleHog base == head olduğu için sıfır dosya tarıyordu — iş yeşildi.
- Slither forge bulunamadığı için anında çöküyordu; continue-on-error bunu gizliyordu.
- slither . | tee rapor.txt — pipefail olmadan çıkış kodu tee'den gelir; Slither düşse bile kapı açık kalırdı.

11.4 Alınan yapısal önlemler

Bulguları tek tek düzeltmek yetmezdi; aynı sınıfın geri gelmesini engelleyecek mekanizmalar kuruldu:

Önem	Mantığı
Belge kod blokları kaynaktan üretiliyor	Elle güncellenen bir alıntı eninde sonunda kayar. 23 blok artık docs/kod_bloklari_senkron.py ile üretiliyor ve --check CI kapısı sapmayı yakalıyor.
Arayüz alanları şemaya karşı doğrulanıyor	Arayüz var olmayan bir alana bağlanamaz; 38 alan her koşulda kontrol ediliyor.
Sınırlar paneli testle korunuyor	Bir sınırı silmek yapıyı kırar. Panel gizlendikten sonra, onu açan bağlantının varlığı da test ediliyor.
Negatif kontrol alışkanlığı	Yeni bir koruma yazıldığında, korumanın gerçekten kırıldığı görülmeden kabul edilmiyor. Bu oturumda birkaç kez bir testin aslında hiçbir şeyi yakalamadığı böyle bulundu.

11.5 Sonradan bulunan üç kritik hata

Denetim listesinde olmayan ama canlı ağda sistemi tamamen çalışmaz hâle getirecek üç hata sonradan bulundu:

Hata	Sonucu
Ön-fonlama gas: 2300 ile yapılıyordu	Gerçek EntryPoint'in receive()'ı mevduat muhasebesi için depolamaya yazar (~20.000+ gaz). 2.300 gaz bir SSTORE'a yetmez ⇒ çağrı her zaman başarısız ⇒ her işlem revert eder. Hesap canlı ağda hiçbir işlemi tamamlayamazdı.
validateUserOp doğrulama sırasında depolamaya yazıyordu	ERC-7562 ihlali. Birçok bundler böyle bir işlemi mempool'a hiç almaz; ayrıca ucuz bir depolama şişirme (DoS) vektörü.
Gösterilen iz, kanıtlanan iz değildi	Sahne jüriye gösterilen tablo STARK'ın kanıtladığı tablo değildi — biri u128, diğeri alan aritmetiği kullanıyordu.

12. Çalıştırma kılavuzu

12.1 Gereksinimler

- Rust (rustup ile) — ZK prover'ı derlemek için
- Python 3.10+ — AI katmanı ve API
- Foundry (isteğe bağlı) — sözleşme testleri için

12.2 Sıfırdan kurulum

```
git clone --depth 1 https://github.com/yorulmazkagan/CryptoTEK.git
cd CryptoTEK

# 1) Rust prover – ZORUNLU, API bu ikiliyi çağırıyor
cd Q-Adaptive-ZK && cargo build --release && cd ..

# 2) Python ortamı
cd Q-Adaptive-AI
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 3) Sunucu
python3 run_server.py
```

Arayüz: <http://127.0.0.1:8000/ui/> — başlangıç yaklaşık 18 saniye sürer (ONNX yükleniyor). «Sunucu isteklere hazır.» satırını görmeden tarayıcıyı açarsanız «SUNUCUYA BAĞLANILAMADI» bandını görürsünüz.

--depth 1 bilinçli: deponun geçmişi büyük sunum dosyaları yüzünden 177 MB'a çıkmış durumda ve tam klonlama ağ kesintilerinde düşebiliyor.

12.3 Her iddiayı doğrulama

```
# Rust: 61 test
cd Q-Adaptive-ZK && cargo test && cargo clippy --all-targets -- -D warnings

# Solidity: 146 test (4'ü fork – sır yoksa atlanır)
cd Q-Adaptive-Contracts && forge test -vv
forge coverage --report summary

# Katmanlar arası eşitlik – gerçek Rust ikilisini çalıştırır
python3 Q-Adaptive-AI/test_layer_parity.py

# Attestation kriptosu: bilinen Keccak-256 / ECDSA vektörleri
python3 Q-Adaptive-AI/test_attestation.py

# API <-> arayüz sözleşmesi
python3 Q-Adaptive-AI/test_api_contract.py

# Belge kod blokları kaynakla eşleşiyor mu
python3 docs/kod_bloklari_senkron.py --check

# Tek koşu: ölçümleri elle görün
```

```
./Q-Adaptive-ZK/target/release/q-adaptive-zk \
  --risk-score 92.4 --tau 75.0 --baseline 44 \
  --user-op-hash 0xdeadbeefcafebabe \
  --epoch-ns 1700000000000000000 --run-id demo
```

Herhangi bir komut kırılırsa, bu belgedeki ilgili iddia geçersizdir. Kasıtlı olarak böyledir: iddialar testlere bağlıdır, yorumlara değil.

12.4 Konuşlandırma (yapılmadı, ama hazır)

```
cd Q-Adaptive-Contracts
cp .env.example .env && $EDITOR .env && source .env

# Kuru koşu – hiçbir işlem yayınlanmaz
forge script script/Deploy.s.sol:Deploy --rpc-url "$RPC_URL"

# Gerçek konuşlandırma
forge script script/Deploy.s.sol:Deploy --rpc-url "$RPC_URL" --broadcast
```

Betik, konuşlandırmadan önce EntryPoint adresinde gerçekten baytkod olduğunu doğrular ve sonra bağlantıları zincirden geri okur. Bu kontrol olmasaydı yanlış ağa konuşlandırma «başarılı» görünürdü: üç sözleşme iner, hiçbiri çalışmaz.

13. Sözlük

Terim	Tanım
AIR	Algebraic Intermediate Representation. STARK izinin sağlaması gereken cebirsel kısıtlar.
BLAKE3	Hızlı kriptografik hash fonksiyonu. Projede ρ' ve kısa tohum türetiminde kullanılır.
Bundler	ERC-4337'de UserOperation'ları toplayıp EntryPoint'e gönderen aktör.
Calldata	Zincire yazılan işlem verisi. Bayt başına ücretlendirilir.
Dilithium	ML-DSA'nın NIST yarışmasındaki adı (CRYSTALS-Dilithium).
Drainer	Özel anahtarı ele geçirip cüzdanı boşaltan saldırı türü.
ECDSA	Eliptik eğri tabanlı imza şeması. Kuantum karşısında kırık.
EntryPoint	ERC-4337 akışını yöneten tekil sözleşme. v0.7 tüm ağlarda aynı adreste.
ERC-4337	Hesap soyutlama standardı. Cüzdanı akıllı sözleşme yapar.
ERC-7562	Doğrulama sırasında nelerin yasak olduğunu belirleyen kural seti.
EVM	Ethereum Sanal Makinesi. Zincir üstünde program çalıştırır.
f128	Winterfell'in 128-bit asal alanı. STARK aritmetiği burada yapılır.
FIPS 204	ML-DSA'nın NIST standardı.
FRI	Fast Reed-Solomon IOP of Proximity. STARK'ın düşük derece testi bileşeni.
Fuzz test	Rastgele üretilmiş girdilerle çalıştırılan test.
Gaz	EVM'de yapılan işin ücreti.
Grover algoritması	Kuantum arama algoritması. Hash güvenliğini yarıya indirir, kırmaz.
Guardian	Zincir dışı kararı imzalayıp zincire taşıyan aktör.
Isolation Forest	Etiketsiz anomali tespiti algoritması.
Kafes (lattice)	Uzayda düzenli dizilmiş nokta kümesi. Post-kuantum kriptografinin temeli.
ML-DSA	Module-Lattice-Based Digital Signature Algorithm. Post-kuantum imza standardı.
MLWE	Module Learning With Errors. ML-DSA'nın dayandığı zor problem.
MTD	Moving Target Defense. Saldırganın keşif yüzeyini sürekli değiştirme stratejisi.
ONNX	Eğitilmiş modelleri taşınabilir biçimde saklama standardı.
Paymaster	Kullanıcının gazını ödeyen ERC-4337 sözleşmesi.
Platt ölçekleme	Ham model skorunu olasılığa kalibre etme yöntemi.
Post-kuantum	Kuantum bilgisayarların çözemediği problemlere dayanan kriptografi.
Reddetme örnekleme	Yanlı olmayan rastgele değer üretmek için aralık dışı örnekleri atma yöntemi.
ρ' (rho-prime)	32 baytlık rotasyon tohumu. Hem ML-DSA anahtarını hem kafes izini besler.
SHAKE-128	İstenilen uzunlukta çıktı veren hash (XOF).
Shor algoritması	Kuantum algoritması. ECDSA ve RSA'yı kırar.

Terim	Tanım
Slither	Solidity için statik zafiyet analiz aracı.
STARK	Scalable Transparent ARgument of Knowledge. Şeffaf, post-kuantum varsayımlı kanıt sistemi.
$\tau(t)$ (tau)	Dinamik anomali eşiği. Kayan pencere varyansından hesaplanır.
UserOperation	ERC-4337'de kullanıcının niyetini taşıyan yapı.
Winterfell	Rust için STARK kanıt kütüphanesi.
XOF	Extendable-output function. İstenilen uzunlukta çıktı veren hash.
ZK (sıfır bilgi)	Bir iddiayı, iddia dışında bilgi sızdırmadan kanıtlama.